

Autonomous Navigation in Unknown Environments Using Model Predictive Path Integral Control

Ahmad Rahmanijavinani
a.rahmani1873@gmail.com

Abstract—This report details a personal project exploring autonomous navigation without the use of an explicit global path planner. A Model Predictive Path Integral (MPPI) controller is implemented to handle local planning and control simultaneously. Using a 2nd-order unicycle kinematic model, the system samples thousands of potential trajectories parallelized via JAX. The cost function is designed to handle highly non-convex environments by penalizing collisions and encouraging exploration through a line-of-sight perception cost. Simulation results demonstrate successful navigation through maze and pole-field environments.

Index Terms—MPPI, Autonomous Navigation, Unicycle Dynamics, JAX, Optimal Control

I. INTRODUCTION

Traditional autonomous navigation pipelines often separate the problem into global path planning (e.g., A*, RRT) and local trajectory tracking (e.g., PID, LQR, or Model Predictive Control). While effective, this decoupling can struggle in partially observable environments where the global map is unknown.

This project explores an alternative approach: using a purely sampling-based receding horizon controller to navigate to a goal. By utilizing Model Predictive Path Integral (MPPI) control, the system can navigate unknown environments without prior map knowledge. MPPI evaluates thousands of forward-simulated trajectories, allowing the robot to implicitly plan around obstacles without a predefined global path.

II. DYNAMICAL SYSTEM

The system under control is modeled using 2nd-order unicycle kinematics. The state vector at time k is defined as $\mathbf{x}_k = [x_k, y_k, \theta_k, v_k]^T$, representing the 2D position, heading angle, and linear velocity. The control input is $\mathbf{u}_k = [a_k, \omega_k]^T$, representing linear acceleration and angular velocity.

The discrete-time update equations using Euler integration with a time step Δt are:

$$x_{k+1} = x_k + v_k \cos(\theta_k) \Delta t \quad (1)$$

$$y_{k+1} = y_k + v_k \sin(\theta_k) \Delta t \quad (2)$$

$$\theta_{k+1} = \theta_k + \omega_k \Delta t \quad (3)$$

$$v_{k+1} = \text{clip}(v_k + a_k \Delta t, -v_{\max}, v_{\max}) \quad (4)$$

The hardware limits of the system are enforced by clipping the control inputs such that $a_k \in [-a_{\max}, a_{\max}]$ and $\omega_k \in [-\omega_{\max}, \omega_{\max}]$.

III. MPPI FORMULATION

Navigating through obstacles with a limited field of view creates a highly non-convex optimization landscape. Standard gradient-based solvers often get stuck in local minima in these scenarios (e.g., driving straight into a U-shaped wall). MPPI bypasses this by using Monte Carlo sampling.

A. Sampling and Weight Update

At each time step, we generate N random control sequences by injecting Gaussian noise into a nominal control sequence:

$$\mathbf{u}_{i,k} = \mathbf{u}_{\text{nom},k} + \mathcal{N}(0, \Sigma) \quad (5)$$

These sequences are rolled out using the unicycle dynamics to generate N trajectories. A cost $S(\mathbf{V}_i)$ is evaluated for each trajectory i . The optimal control sequence is then calculated as a cost-weighted average of the sampled inputs:

$$\mathbf{u}_k^* = \mathbf{u}_{\text{nom},k} + \sum_{i=1}^N w_i \delta \mathbf{u}_{i,k} \quad (6)$$

where the weights w_i are computed using a softmax-like function:

$$w_i = \frac{\exp\left(-\frac{1}{\lambda}(S(\mathbf{V}_i) - \beta)\right)}{\sum_{j=1}^N \exp\left(-\frac{1}{\lambda}(S(\mathbf{V}_j) - \beta)\right)} \quad (7)$$

Here, λ is the temperature parameter tuning the algorithm's selectivity, and $\beta = \min_i S(\mathbf{V}_i)$ is used for numerical stability.

B. Cost Function

The success of the planner heavily relies on the cost function. The total cost consists of three main (detailed formulation present in the code):

- **Goal Cost:** Penalizes the squared Euclidean distance from the trajectory states to the target goal, encouraging forward progress.
- **Obstacle Cost:** Applies a massive penalty if any state in the trajectory hits a known wall or goes out of map bounds.
- **Perception Cost:** If the goal is not in direct line-of-sight (LOS), the algorithm traces rays from the trajectory endpoints. It heavily penalizes paths that dead-end into known walls and rewards paths that lead to unexplored/unknown space, naturally encouraging the robot to explore around corners.

IV. SIMULATION SETUP

The simulation is implemented in Python. To achieve the performance necessary for MPPI (rolling out thousands of trajectories per step), the code utilizes JAX. Using `jax.vmap` allows parallel trajectory rollouts, and `jax.jit` compiles the dynamic steps and cost evaluations into optimized machine code.

The environment is a 2D grid with a resolution of 0.1 m. The vehicle starts with a blank belief map and updates it iteratively using a simulated sensor with a 90° field of view and a limited sensing radius. The MPPI controller uses $N = 1000$ samples and a prediction horizon of $H = 50$ steps with $\Delta t = 0.1$ s.

V. RESULTS

The algorithm was tested on two custom maps: a dense pole field and a structured maze. In both scenarios, the vehicle had to build its belief map iteratively while avoiding collisions.

Fig. 1 shows the results in the pole environment. The red arrows represent the vehicle’s heading over time, and the blue line denotes the executed trajectory. The control plots (velocity, acceleration, angular velocity) demonstrate how MPPI outputs aggressive but safe maneuvers to dodge the poles.

Fig. 2 illustrates the performance in a maze. Without a global planner, the vehicle might normally get stuck in local minima. However, the combination of MPPI sampling and the ray-tracing perception cost allows the vehicle to safely navigate the winding corridors, adjusting its belief map dynamically.

VI. CONCLUSION

This project demonstrates that MPPI is a highly effective method for integrated planning and control in unknown environments. By leveraging JAX for parallelized sampling, the controller can evaluate complex cost functions. The 2nd-order unicycle model ensures the resulting trajectories are physically feasible for a wheeled or flying robot. I would like to extend the work to consider more complex models and verification in high-fidelity simulators before hardware implementation.

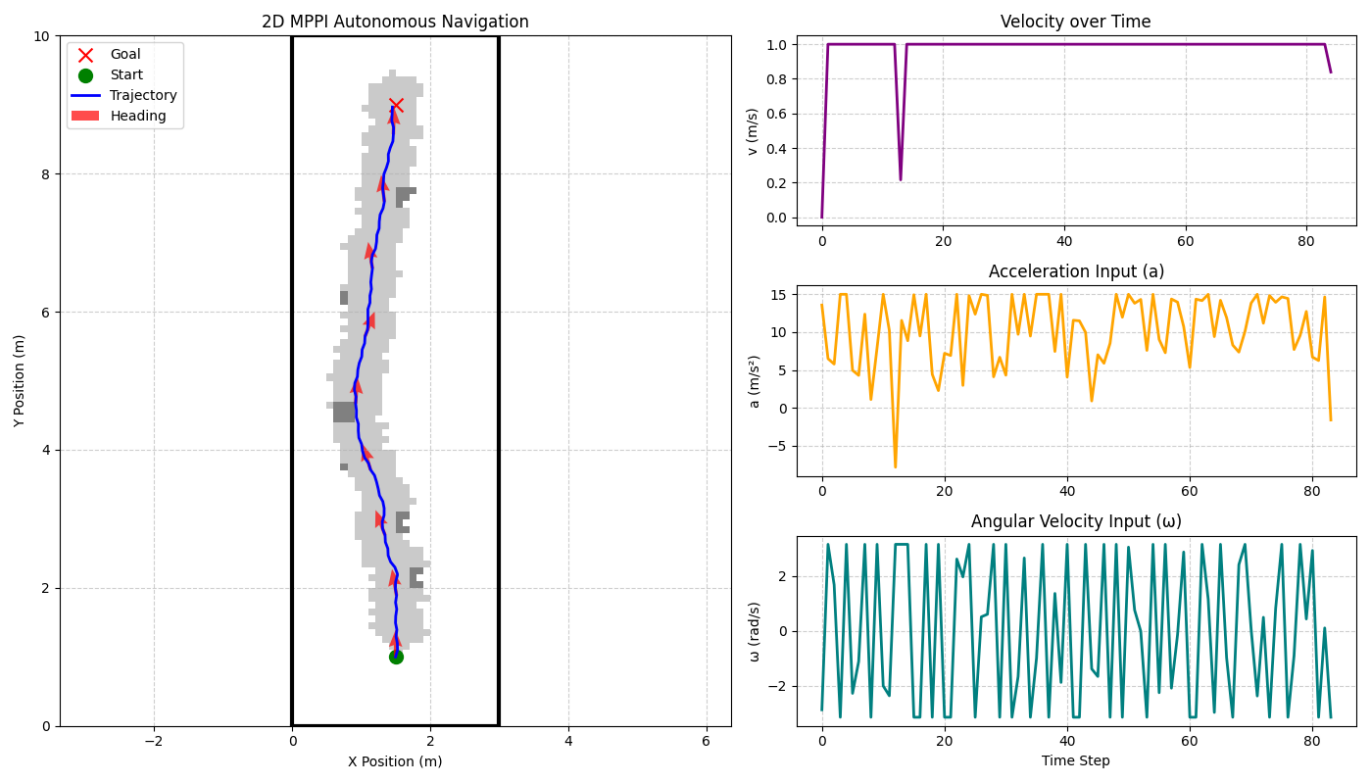


Fig. 1. Trajectory and control inputs for the pole field environment. The vehicle weaves through isolated obstacles successfully.

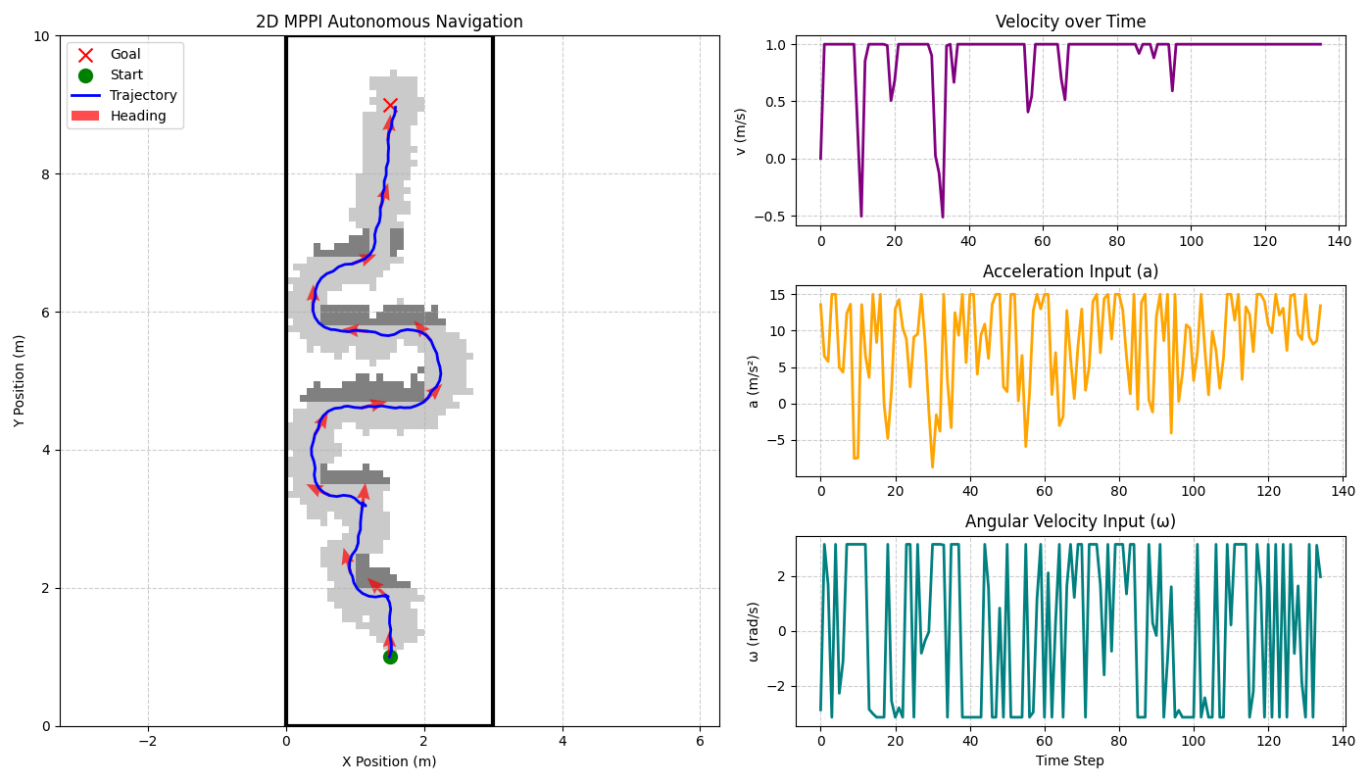


Fig. 2. Trajectory and control inputs for the maze environment. The perception cost helps the vehicle navigate around tight corners without a global planner.